📋 Informations

- **Durée** : 1h30 (1h HTML + 30 min CSS)
- **Séance** : 2 (10 mars)
- **Prérequis** : Cours théorique de la séance 1 (balises sémantiques, sélecteurs CSS, box model)
- **Outils** : VS Code, Live Server, navigateur moderne
- **Code de base**: https://github.com/zenocode-org/mira-im4ddw/tree/main/repos/seance-02-tp

---

But général du TP

Tu vas créer **une page portfolio personnelle** en HTML et CSS. À la fin, tu maîtriseras la structure sémantique d'une page web, le lien entre HTML et feuille de style externe, et l'application des propriétés CSS de base (couleurs, typographie, espacements, Flexbox). Ce projet servira de base pour la séance 3 où on le rendra responsive.

💡 Concepts théoriques préalables

Avant de coder, assure-toi de comprendre :

HTML :

- Les balises sémantiques : `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>` — chacune a une signification précise pour le contenu qu'elle entoure
- La différence entre `id` (unique, pour les ancres et le JS) et `class` (réutilisable, pour le style). **En CSS : privilégie les classes ; réserve les `id` aux ancres de navigation.**
- Les liens internes (`href="#section"`) pointent vers un `id` sur la même page

CSS :

- Le fichier CSS externe est lié dans le `<head>` via `<link rel="stylesheet" href="...">`
- Les chemins sont relatifs : `style.css` si le fichier est au même niveau que `index.html`
- Le sélecteur `*` cible tous les éléments ; les classes avec `.nom` ; les ID avec `#nom` — utilise `.nom` pour le style, `#nom` uniquement si nécessaire (ancres)

Serveur local :

- En double-cliquant sur `index.html`, le navigateur ouvre via `file://` — ça marche pour du HTML simple, mais certains comportements peuvent différer
- **Live Server** (extension VS Code) lance un petit serveur local : clic droit sur `index.html` → "Open with Live Server". Les rechargements sont plus fiables et le rechargement automatique facilite le travail

Questions à te poser :

- Quelle balise contient le contenu principal de la page ? (indice : une seule par page)
- Comment faire pour que plusieurs éléments aient le même style ? (classe ou ID ?)

🌐 Vision globale : sites à une ou plusieurs pages

Pour ce TP, tu crées **une seule page** (`index.html`). En pratique, un site peut avoir plusieurs pages :

```
mon-site/
├── index.html        ← Page d'accueil (convention)
├── projets.html      ← Autre page
├── style.css         ← CSS partagé entre toutes les pages
└── images/
```

- **Liens entre pages** : `<a href="projets.html">Mes projets</a>` charge une autre page
- **Liens internes (ancres)** : `href="#contact"` défile sur la même page

On restera sur une page unique pour ce TP. On verra les sites multi-pages plus tard (séance 3 ou 6).

📁 Structure de fichiers

Crée un dossier `portfolio/` avec :

```
portfolio/
├── index.html
├── style.css
└── images/
```

🔨 Partie 1 : Structure HTML (1h)

Exercice 1.1 : Base du document

But : Avoir un document HTML valide avec les métadonnées essentielles et le lien vers la feuille CSS.

Ce que tu dois faire :

- Créer `index.html` avec la structure minimale d'une page HTML5
- Inclure dans le `<head>` : encodage UTF-8, viewport pour le responsive, titre de la page, lien vers `style.css`
- Laisser le `<body>` vide pour l'instant

Comment l'aborder :

- Rappel : `<!DOCTYPE html>`, `<html lang="fr">`, `<head>`, `<body>`
- Dans le `<head>` : `<meta charset="UTF-8">`, `<meta name="viewport" content="width=device-width, initial-scale=1.0">`, `<title>`, `<link rel="stylesheet" href="style.css">`
- Le `lang="fr"` aide les lecteurs d'écran et le SEO

Documentation :

- MDN : Éléments HTML de base
- W3Schools : HTML Head

À toi de jouer !

Résultat attendu : Une page blanche qui s'ouvre dans le navigateur. Si tu ouvres les DevTools (F12) → onglet Elements, tu vois une structure `<html>`, `<head>`, `<body>` propre.

Questions à te poser : Où doit-on placer le lien vers le CSS — dans le head ou le body ? Pourquoi ?

Exercice 1.2 : Header et navigation

But : Créer l'en-tête de la page avec ton identité, une photo de profil et un menu de navigation.

Ce que tu dois faire :

- Ajouter un `<header>` contenant : une photo de profil (en haut), un titre principal (ton prénom et nom), un sous-titre (ex. "Étudiant·e en développement web"), et une navigation avec une liste de liens

- Les liens doivent pointer vers : À propos, Compétences, Projets, Contact (sections que tu créeras ensuite)
- Intégrer une image de profil : fichier dans `images/` (ex. `profil.jpg`) ou URL externe. L'attribut `alt` est obligatoire (accessibilité et SEO)

Comment l'aborder :

- `<img src="images/profil.jpg" alt="Photo de profil" class="photo-profil">` — place l'image au début du header, avant le titre
- Si tu n'as pas d'image : utilise une photo de profil générée (ex. UI Faces ou une image placeholder)
- Pour les liens internes (défilement dans la page), utilise `href="#nom-de-section"` — le `id` correspondant sera sur chaque `<section>`
- Structure typique : `<nav>` contient `<ul>` qui contient plusieurs `<li>`, chaque `<li>` contient un `<a>`
- Personnalise le contenu (prénom, nom, sous-titre)

Documentation :

- MDN : Élément img
- MDN : Élément nav
- MDN : Liens HTML

À toi de jouer !

Résultat attendu : Un header visible en haut de la page avec une photo de profil, ton nom et des liens cliquables. Les liens ne mèneront nulle part pour l'instant (les sections n'existent pas encore).

Questions à te poser : Pourquoi l'attribut `alt` est-il important même si l'image s'affiche correctement ? Quelle est la différence entre `<a href="page.html">` et `<a href="#section">` ?

Exercice 1.3 : Section "À propos"

But : Créer le contenu principal avec ta présentation.

Ce que tu dois faire :

- Ajouter un `<main>` contenant une première section
- Cette section doit avoir un `id` correspondant au lien de navigation "À propos"
- Contenu : un titre de section (h2) et 2 à 3 paragraphes te présentant (passion, parcours, motivations)

Comment l'aborder :

- La balise `<main>` ne doit apparaître qu'une fois par page : c'est le contenu principal
- Chaque `<section>` peut avoir un `id` pour les ancres : `id="a-propos"` (sans espace, sans accents si possible pour éviter les soucis)
- Utilise `<p>` pour les paragraphes, `<h2>` pour le titre de section

À toi de jouer !

Résultat attendu : Une section visible avec ton texte. Le lien "À propos" dans la nav doit maintenant faire défiler vers cette section.

Questions à te poser : Pourquoi l'`id` de la section doit-il correspondre au `href` du lien (sans le `#`) ?

Exercice 1.4 : Section "Compétences"

But : Lister tes compétences par catégorie.

Ce que tu dois faire :

- Ajouter une section "Compétences" avec un `id` cohérent avec la nav (pour l'ancre) et la classe `section-competences` (pour le CSS)
- Organiser les compétences en 2 ou 3 catégories (ex. Front-end, Back-end, Outils) — utilise `<article class="competence-card">` pour chaque catégorie avec un `<h3>` et une liste `<ul>` / `<li>`

Comment l'aborder :

- Les compétences peuvent être fictives pour l'exercice ; "en cours d'apprentissage" est parfaitement acceptable

Documentation :

- MDN : Élément article

À toi de jouer !

Résultat attendu : Une section compétences avec des listes à puces par catégorie.

Exercice 1.5 : Section "Projets"

But : Présenter 2 à 3 projets (réels ou fictifs) de manière structurée.

Ce que tu dois faire :

- Ajouter une section "Projets" avec un `id` cohérent
- Pour chaque projet : un titre (h3), une courte description
- Utilise la classe `projet` sur chaque bloc de projet (tu la styleras en CSS)

Comment l'aborder :

- `<article class="projet">` pour chaque projet
- La classe permet d'appliquer le même style à plusieurs éléments
- Tu peux inventer des projets si tu n'en as pas : ex. "Portfolio responsive", "To-do list en JS", "Mini-jeu"

À toi de jouer !

Résultat attendu : Une section avec 2-3 blocs projet. La classe `.projet` sera utilisée en CSS pour un style commun.

Questions à te poser : Pourquoi utiliser une `class` plutôt qu'un `id` pour les projets ?

Exercice 1.6 : Section "Contact" (formulaire)

But : Créer un formulaire de contact avec les champs essentiels.

Ce que tu dois faire :

- Ajouter une section "Contact" avec un formulaire
- Champs : Nom (texte), Email (email), Message (textarea)
- Un bouton "Envoyer"

Comment l'aborder :

- `<form>` contient les champs.
- Le formulaire doit envoyer les données vers un **endpoint volontairement inexistant** (par exemple `/contact` ou `/api/contact`).
- Chaque champ doit avoir :
  - un `<label for="id-du-champ">`
  - un `<input>` ou `<textarea>` avec un `id` et un `name` correspondants
- Ajouter `required` sur les champs obligatoires pour utiliser la validation HTML5.
- Types utiles : `type="text"`, `type="email"`.

Même si l'endpoint n'existe pas, le navigateur va **quand même essayer d'envoyer la requête**.

👉 Après avoir cliqué sur **Envoyer**, ouvre les **DevTools du navigateur** et observe ce qu'il se passe.

Documentation :

- MDN : Formulaires HTML

Résultat attendu :

- Un formulaire avec 3 champs et un bouton.
- Lorsque tu cliques sur **Envoyer**, une tentative d'envoi est faite vers l'endpoint (qui échouera, c'est normal).

Questions à te poser :

1. Pourquoi associe-t-on un `label` à un champ via `for` et `id` ?
2. Que se passe-t-il dans le navigateur quand tu cliques sur **Envoyer** ?
3. Peux-tu trouver **où la requête du formulaire apparaît dans les DevTools** et quelles informations sont envoyées ?

Exercice 1.7 : Footer

But : Fermer la page avec un pied de page et des liens externes.

Ce que tu dois faire :

- Ajouter un `<footer>` après `</main>`, avant `</body>`
- Contenu : copyright (ton nom, année), liens vers GitHub, LinkedIn, email

Comment l'aborder :

- `©` affiche le symbole ©
- Pour un email cliquable : `<a href="mailto:ton@email.com">Email</a>`
- `target="_blank"` ouvre les liens externes dans un nouvel onglet

À toi de jouer !

Résultat attendu : Un footer visible en bas de page avec les liens. Tu peux mettre des `#` temporairement si tu n'as pas encore de profils.

Documentation :

- W3School: HTML Symbols
- W3School: Address Tag

✅ Checkpoint Partie 1

Avant de passer au CSS, vérifie :

- [ ] Un fichier HTML valide avec header, main (4 sections), footer
- [ ] Toutes les sections ont un `id` correspondant aux liens de la nav
- [ ] Au moins une image intégrée
- [ ] Le lien "À propos" (et les autres) fait défiler vers la bonne section

Test : Ouvre avec Live Server. Tout le contenu s'affiche. Les liens de navigation fonctionnent. Pas de style encore — c'est normal.

🎨 Partie 2 : CSS de base (30 min)

Crée `style.css` dans le dossier `portfolio/`. Toutes les modifications de style se feront dans ce fichier.

Approche Mobile First

On écrit d'abord les styles pour les **petits écrans (mobile)**, puis on améliore pour les écrans plus larges avec des media queries `@media (min-width: ...)`.

Consigne pratique : Dans chaque exercice, commence par styler pour mobile (fenêtre étroite ou DevTools en mode mobile). Ensuite, ajoute des media queries pour adapter le rendu sur tablette puis sur desktop.

Media queries : largeurs standards en pixels

Les breakpoints utilisent des **largeurs en pixels** qui correspondent aux tailles typiques des appareils : mobile (petit écran), tablette (écran moyen), desktop (grand écran). Ces valeurs sont des standards de l'industrie (Bootstrap, W3Schools, etc.) — elles correspondent aux largeurs réelles des viewports sur lesquels les utilisateurs consultent le web.

Ta mission : Ton portfolio doit s'adapter correctement sur **mobile, tablette et desktop**. Les valeurs précises des breakpoints ne te sont pas données : **tu dois les rechercher**. Consulte par exemple W3Schools : CSS3 Media Queries - Examples pour voir quelles largeurs sont couramment utilisées (ex. 600px, 768px, 992px...) et pourquoi. Choisis des breakpoints cohérents pour que la nav et la section compétences s'affichent correctement sur chaque type d'écran.

Astuce : Utilise les DevTools (F12) → icône mobile/tablette pour tester différentes largeurs et vérifier que tes media queries se déclenchent bien.

Unités à privilégier

- **`rem`** pour les espacements, tailles de police (ex. `1rem`, `0.25rem`)
- **`ch`** pour la largeur de contenu lisible (ex. `max-width: 65ch`)
- **`%`** **ou** **`vw`** pour les largeurs fluides
- **`px`** pour les media queries (breakpoints standards) et le `border-radius`

Exercice 2.1 : Reset et styles de base

But : Partir d'une base propre et définir la typographie et les couleurs globales.

Ce que tu dois faire :

- Réinitialiser les marges et paddings par défaut du navigateur
- Appliquer `box-sizing: border-box` à tous les éléments
- Sur le `body` : police, interligne, couleur de texte, couleur de fond (utilise `rem` pour les tailles de police et interlignes)

Comment l'aborder :

- Le sélecteur `*` cible tous les éléments
- `box-sizing: border-box` inclut padding et border dans la largeur/hauteur — plus facile pour les calculs
- Choisis une police web-safe (Arial, Georgia) ou une Google Font si tu veux aller plus loin
- Une couleur de fond légère (gris clair) et un texte foncé améliorent la lisibilité

Documentation :

- W3School: Hover pseudo class
- MDN : Box-sizing

À toi de jouer !

Résultat attendu : La page n'a plus les marges par défaut du navigateur. Le texte et le fond ont une apparence cohérente.

Exercice 2.2 : Header

But : Donner une identité visuelle forte au header.

Ce que tu dois faire :

- Fond coloré (sombre), texte clair, centrage
- Titre (h1) plus grand, sous-titre un peu plus petit

Comment l'aborder :

- `background-color`, `color`, `text-align: center`, `padding`
- `header h1` et `header p` pour cibler les enfants du header
- Utilise Coolors ou ColorHunt pour trouver une palette si tu veux

À toi de jouer !

Résultat attendu : Un header visuellement distinct du reste de la page. (Sur grands écrans, tu peux augmenter le padding avec une media query si tu le souhaites.)

Exercice 2.3 : Navigation (Flexbox)

But : Afficher les liens de la nav de façon adaptée à chaque taille d'écran, avec un effet au survol.

Ce que tu dois faire :

- Retirer les puces de la liste
- **Mobile (styles de base) :** Liens empilés verticalement ou en ligne avec `flex-wrap: wrap` pour éviter le débordement
- **Desktop (media query) :** Liens en ligne horizontale, bien espacés — utilise une largeur en pixels issue de ta recherche
- Changer la couleur au survol (`:hover`)

Comment l'aborder :

- Flexbox s'applique sur le **parent** : `nav ul { display: flex; }` — les `<li>` deviennent des éléments flex
- `list-style: none` retire les puces
- En base : `flex-direction: column` ou `flex-wrap: wrap` selon ton choix pour mobile
- Ajoute une media query `@media (min-width: XXXpx)` avec la largeur adaptée au desktop (recherche les standards)
- `nav a:hover { color: ... }` pour l'effet au survol
- `transition: color 0.3s` pour une animation douce

Documentation :

- CSS-Tricks : Guide Flexbox

À toi de jouer !

Résultat attendu : Sur mobile, les liens sont lisibles (empilés ou wrap). Sur grand écran, ils sont alignés en ligne et réagissent au survol.

Questions à te poser : Sur quel élément appliques-tu `display: flex` — le `nav` ou le `ul` ?

Exercice 2.4 : Main et sections

But : Centrer le contenu, limiter sa largeur, et donner du relief aux sections.

Ce que tu dois faire :

- Contenu principal centré avec une largeur max fluide (ex. `min(65ch, 90vw)`) pour une lecture confortable
- Chaque section : fond blanc, padding, marge en bas, coins arrondis, ombre légère
- Titres de section (h2) : style distinct (couleur, bordure en bas ?)

Comment l'aborder :

- `65ch` ≈ largeur optimale d'une ligne de texte ; `90vw` évite que le contenu touche les bords sur mobile
- `max-width` + `margin: 0 auto` (ou `margin: 2rem auto`) pour centrer un bloc
- `box-shadow: 0 2px 5px rgba(0,0,0,0.1)` pour une ombre subtile
- `border-radius: 5px` (ou autre valeur en px) pour les coins arrondis

À toi de jouer !

Résultat attendu : Le contenu est centré, les sections ressemblent à des cartes.

Exercice 2.5 : Section Compétences (CSS Grid)

But : Afficher les articles de compétences de façon adaptée à chaque taille d'écran.

Ce que tu dois faire :

- **Mobile (styles de base) :** Une colonne — les 3 articles s'empilent verticalement
- **Tablette (première media query) :** 2 colonnes pour plus d'espace par carte — cherche la largeur adaptée
- **Desktop (deuxième media query) :** 3 colonnes avec `minmax(10rem, 1fr)` pour éviter que les cartes soient trop étroites — cherche la largeur adaptée
- Chaque article a un fond légèrement différent, du padding

- Le titre (h2) de la section doit occuper toute la largeur sur une ligne

Comment l'aborder :

- **Ne pas utiliser d'`id` pour le CSS** — réserve les `id` aux ancres de navigation (`href="#competences"`). Utilise une **classe** pour le style (ex. `class="section-competences"` sur la section, `class="competence-card"` sur chaque article).
- CSS Grid : `display: grid` sur `.section-competences`
- En base : `grid-template-columns: 1fr` (une colonne)
- Le h2 doit s'étendre sur toute la largeur : `.section-competences h2 { grid-column: 1 / -1; }`
- Deux media queries en pixels (largeurs à trouver via W3Schools) :
  - une pour tablette : `repeat(2, 1fr)` — 2 colonnes (cartes moins serrées)
  - une pour desktop : `repeat(3, minmax(10rem, 1fr))` — 3 colonnes avec largeur minimale par carte
- `gap` pour l'espacement
- Classe `.competence-card` sur chaque article : styles (fond, padding) via `.competence-card`

Pourquoi deux breakpoints ? Un seul breakpoint avec 3 colonnes peut rendre les cartes trop étroites sur tablette. Un passage par 2 colonnes sur tablette donne plus d'espace par carte avant d'arriver à 3 colonnes sur grand écran. Le `minmax(10rem, 1fr)` évite le rétrécissement excessif des cartes.

Pourquoi des classes et pas des `id` ? Les classes sont réutilisables et n'imposent pas d'unicité — elles décrivent le rôle de l'élément (section en grille, carte de compétence). L'`id` reste sur la section uniquement pour que le lien `#competences` fonctionne.

Pourquoi Grid plutôt que Flexbox ? Avec 3 cartes fixes, `repeat(3, 1fr)` donne directement 3 colonnes sans calcul de flex-basis ni sélecteurs de position (`nth-child`).

Documentation :

- MDN : CSS Grid Layout
- CSS-Tricks : Guide Grid

À toi de jouer !

Résultat attendu : Les catégories de compétences s'affichent en 1 colonne sur mobile, 2 colonnes sur tablette, et 3 colonnes sur desktop. Les cartes ne sont pas trop étroites. Vérifie avec les DevTools que chaque breakpoint se déclenche correctement.

Exercice 2.6 : Projets (classe `.projet`)

But : Donner un style visuel distinct à chaque bloc projet.

Ce que tu dois faire :

- Créer un accent visuel (bordure gauche colorée, par exemple)
- Espacement entre les projets

Comment l'aborder :

- `border-left: 4px solid #couleur` ou `border-left: 0.25rem solid #couleur` sur `.projet`
- `padding-left` pour ne pas coller le texte à la bordure

À toi de jouer !

Résultat attendu : Chaque projet a une barre colorée à gauche.

Exercice 2.7 : Formulaire

But : Rendre le formulaire lisible et agréable à utiliser.

Ce que tu dois faire :

- Labels au-dessus des champs
- Champs en pleine largeur, avec padding, bordure, coins arrondis
- Effet au focus (bordure qui change de couleur)
- Bouton stylé avec couleur de fond et effet au survol

Comment l'aborder :

- `label { display: block; }` pour mettre le label au-dessus du champ
- `input, textarea { width: 100%; padding: ...; border: ...; }` — utilise `rem` pour le padding et les bordures
- `:focus` pour l'état quand on clique dans un champ
- `cursor: pointer` sur le bouton

Documentation :

- MDN : Pseudo-classes

À toi de jouer !

Résultat attendu : Un formulaire propre, lisible, avec des retours visuels au focus et au survol.

Exercice 2.8 : Footer

But : Donner au footer le même style que le header (cohérence visuelle).

Ce que tu dois faire :

- Fond sombre, texte clair, centré
- Liens stylés avec effet au survol

Comment l'aborder :

- Même palette que le header pour l'unité
- `footer a` pour les liens

À toi de jouer !

Résultat attendu : Un footer harmonieux avec le reste de la page.

Exercice 2.9 : Image de profil (optionnel dans le flux)

Si tu as ajouté une photo de profil dans le header, style-la :

- Taille adaptée (ex. `width: 10rem; height: 10rem` ou `min(10rem, 30vw)`)
- `border-radius: 50%` pour un cercle
- Bordure ou ombre pour la faire ressortir

✅ Checkpoint Partie 2

Ta page devrait maintenant :

- [ ] Approche Mobile First : styles de base pour mobile, media queries pour tablette et desktop
- [ ] Media queries avec largeurs en pixels (standards mobile/tablette/desktop)
- [ ] Au moins une media query `min-width` utilisée (nav, compétences ou main)
- [ ] Avoir une palette de couleurs cohérente
- [ ] Utiliser Flexbox pour la nav et Grid pour la section compétences
- [ ] Avoir des espacements lisibles (margin, padding)
- [ ] Être agréable à parcourir

🚀 Pour aller plus loin (facultatif, sans solution fournie)

Défi 1 : Animation au chargement

Fais apparaître les sections avec une légère animation (opacité, translation). Cherche `@keyframes` et `animation` sur MDN.

Défi 2 : Mode sombre

Ajoute un bouton pour basculer entre mode clair et sombre. Le JavaScript pour le toggle viendra à la séance 4 ; tu peux préparer les classes CSS (ex. `.dark-mode`).

Défi 3 : Site multi-pages

Crée `projets.html` et `contact.html` avec le même header/footer. Lie le même `style.css`. Observe la duplication — c'est le problème que résolvent les frameworks (voir TP bonus Astro).

🐛 Aide et débogage

Ma feuille CSS ne s'applique pas

1. Vérifie que `style.css` est dans le même dossier que `index.html`
2. Vérifie le lien : `<link rel="stylesheet" href="style.css">` dans le `<head>`
3. Vide le cache : Ctrl+Shift+R (ou Cmd+Shift+R sur Mac)
4. Ouvre la console (F12) : des erreurs 404 ? Un chemin incorrect ?

Les liens de navigation ne fonctionnent pas

Les `href="#section"` pointent vers un `id`. Vérifie que chaque section a bien `id="a-propos"`, `id="competences"`, etc. (sans le `#` dans l'attribut).

Flexbox ne s'affiche pas comme prévu

Flexbox agit sur les **enfants directs** du conteneur. Si tu mets `display: flex` sur `nav`, ce sont les enfants de `nav` (souvent un `ul`) qui sont affectés. Pour une liste horizontale, c'est souvent `nav ul` qu'on met en flex.

Mobile First / Media queries

Les styles de base s'appliquent à tout. Les media queries `@media (min-width: XXXpx) { ... }` ajoutent ou surchargent des règles pour les écrans plus larges. Si rien ne change en élargissant la fenêtre, vérifie que la media query est bien placée **après** les styles de base, que la condition `min-width` est correcte (en pixels), et que la largeur du viewport dépasse bien ton breakpoint. Utilise les DevTools pour inspecter la largeur actuelle.

L'image ne s'affiche pas

- Vérifie le chemin : `images/profil.jpg` si le fichier est dans le dossier `images/`
- Le chemin est relatif au fichier HTML
- Vérifie l'extension (jpg, png, webp)

📚 Ressources

- **MDN Web Docs** : https://developer.mozilla.org/fr/docs/Web/HTML et https://developer.mozilla.org/fr/docs/Web/CSS
- **MDN : Media queries** : https://developer.mozilla.org/fr/docs/Web/CSS/Media_Queries
- **W3Schools : Media Queries - Examples** : https://www.w3schools.com/css/css3_mediaqueries_ex.asp (largeurs standards pour mobile, tablette, desktop)
- **MDN : Valeurs et unités CSS (rem, ch)** : https://developer.mozilla.org/fr/docs/Web/CSS/CSS_Values_and_Units
- **CSS-Tricks** : https://css-tricks.com/ (Flexbox, Grid)
- **Coolors** : https://coolors.co/ (palettes)
- **Google Fonts** : https://fonts.google.com/

✅ Critères de réussite

HTML :

- [ ] Structure sémantique complète (header, main, sections, footer)
- [ ] Contenu personnalisé
- [ ] Au moins une image avec `alt`
- [ ] Attributs `id` et `class` utilisés à bon escient

CSS :

- [ ] Fichier externe lié
- [ ] Palette cohérente
- [ ] Typographie et espacements lisibles
- [ ] Approche Mobile First avec media queries en pixels (largeurs standards)
- [ ] Flexbox (nav) et Grid (compétences)

Bonus : Transitions, image stylée, palette originale

📥 Rendu

À rendre pour la séance 3 :

- Dossier `portfolio/` (index.html, style.css, images)
- Déposé sur la plateforme de cours

Note : Les solutions complètes sont dans `solutions-tp2-seance.md` — consulte-les après avoir essayé !

Bon courage ! 💪